



Be Persistent.

A Comparison between Relational and Object Oriented Database systems for Object Oriented Application Development

Dirk Bartels
President & CEO

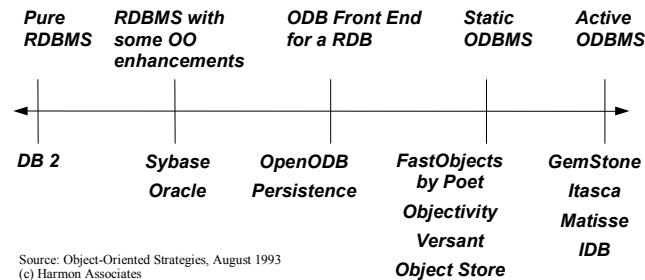
A FastObject™ White Paper

Introduction

This White Paper is intended to clarify the differences between relational and object oriented database systems, especially from the perspective of object oriented application development, using programming languages like Java™ technology or C++. The document is structured into 3 parts:

- Chapter 1 explains the object oriented and the relational model. We are using Java technology and SQL for basic examples.
- Chapter 2 describes the ways and shortcomings of integrating an object oriented application model with a relational database system.
- Chapter 3 gives a short overview on object oriented database systems and their impact on object oriented application development.

There are many different kinds of database systems available, from pure relational database systems (RDBMS) to pure object oriented database systems (ODBMS). When we talk about ODBMS in this paper, we only mean static or active ODBMS. Relational database systems are the alternative, more traditional systems discussed in this paper. ODB front ends might reduce some of the shortcomings that are described later on. However they don't change the underlying data model and provide more a migration path than a new technology.



A different trend can be seen in last 30 years of software development. We have seen fundamental changes in the way we are using and programming a computer. Today object orientation is the leading methodology and technology used in the software industry.

User Interface	Character Terminal	Personal Computer	GUI
Data model	CODASYL	Relational, SQL	Object Oriented
Programming Language	COBOL	C, Pascal	C++, Java, Smalltalk
Mode	3270/VT100	Interactive	Event driven
	'70	'80	'90

The authors hope that this white paper helps you to make your business decisions. Please let us know if you have any comments to this paper. We would greatly appreciate it.

Yours,

Dirk Bartels (dirk.bartels@poet.com)

Relational Databases and object oriented Languages

Relational database systems revolutionized database management systems in the 1980s, and object oriented programming languages were revolutionizing software development in the late 1980s and 1990s. These systems seem to have complementary strengths. Relational database systems are good for managing large amounts of data; object oriented programming languages are good at expressing complex relationships among objects. Relational database systems are good for data retrieval but provide little support for data manipulation; object oriented programming languages are excellent at data manipulation but provide little or no support for data persistence and retrieval. Many people are trying to combine the two in order to manage large amounts of data with complex relationships.

Unfortunately, the relational model and the object model are fundamentally different, and integrating the two is not straightforward. Relational database systems are based on two-dimensional tables in which each item appears as a row. Relationships among the data are expressed by comparing the values stored in these tables. Languages like SQL allow tables to be combined on the fly to express relationships among the data. The object model is based on the tight integration of code and data, flexible data types, hierarchical relationships among data types, and references. Expressing these basic structures in the two-dimensional tables of relational database systems is not trivial, and this is only the first step. The interface between the two systems must correctly handle data manipulation and data retrieval.

The semantic mismatch between object oriented programming languages and relational databases has led to the development of object oriented database systems which directly support the object model. Object oriented database systems are generally much simpler to use in object oriented programs.

This chapter focuses on the shortcomings of relational database systems in object oriented programming environments. These shortcomings might make even small programs more complex and harder to understand. Because of this additional complexity, large and complex applications might become impossible to oversee and to maintain. In order to understand these problems we need to understand the models involved.

The Object Model

The object oriented model is based on objects, which are structures that combine related code and data. The description of an object is contained in its class declaration. The basic principles of objects are called encapsulation, inheritance and polymorphism. We will use invoices as an example to illustrate these points.

Encapsulation

Encapsulation means that code and data are packaged together to form objects, and that the implementation of these objects can be hidden from the rest of the program. For instance, a class declaration for an item might look like this:

```
public class Item {
    // data members
    String    name;
    float    price;
    int      number_in_stock;
    // methods
    public Item() { /*...*/ }
    public int NumAvail() { /*...*/ }
    public int Buy(int n) { /*...*/ }
    public int Sell(int n) { /*...*/ }
    public float TotalPrice() { /*...*/ }
}
```

Packaging the code and data together helps clarify the relationships among them. If the `Buy()` method increases the number of items in stock, it is clear from the above declaration that `number_in_stock` belongs to the `Item` class. However, this detail is hidden from other classes which use `Item`. Because `number_in_stock` is not public it can not be accessed directly except in the methods belonging to the `Item` class. However, the public part of the class declaration includes methods which use `number_in_stock`. These methods may be accessed by any method of other classes.

Inheritance

Inheritance is a powerful mechanism which lets a new class be built using code and data declared in other classes. This allows common features of a set of classes to be expressed in a base class.

Suppose we want to add a new class called `ClothingItem`. All of the code and data defined for `Item` are still relevant, but we need to add sizing information. We can do this by saying that a `ClothingItem` is an `Item` which also has a size. In Java™ this might look something like this:

```
public class ClothingItem extends Item {  
    int size;  
}
```

Because we said that `ClothingItem` is an `Item` it has all the code and data defined for `Item`.

Polymorphism

Suppose you have a variety of `Items`, but the rules for computing their total price differs. For instance, medical items might be exempt from sales tax in an American database system, or a German database system might compute value added tax differently for different items. You still want all `Items` to have a method called `TotalPrice()`, but instead of one `TotalPrice()` method we now want different methods depending on the specific class.

Here is an oversimplified example:

```
public class Item {  
    // data members  
    String    name;  
    float     price;  
    int       number_in_stock;  
    // methods  
    public float TotalPrice() { return price; }  
}  
  
public class ClothingItem extends Item {  
    // data members  
    int       size;  
    // methods  
    public float TotalPrice() { return 1.07 * price; }  
}  
  
public class Medicialtem extends Item {  
}
```

A method which calculates the total price for an invoice can ask each item for its total price without knowing how it is computed for that item. This code will be valid for any `Item` class -- even new classes that are added later!

Object Identity, References among Objects, and Collections

In Java object references are used to establish relationships between objects.

Every object in an object oriented system has its own identity. This object identity does not depend on the data values it contains. In Java two objects are identical if their references are identical, i.e. if their references refer to the same target object. In contrast, the method `isEqual()` evaluates if the data values are identical. Hence there are two different concepts for identity: object identity and value identity.

If an object has a single relationship (1:1) to another object, you simply use a reference to establish this relationship. If an object has a multiple relationship (1:N, M:N) to another object or to a set of other objects, you use container classes or arrays of references to establish these multiple relationships. For instance, if an invoice has a date, a customer, and a set of items we could declare it like this (`SetOfObject` is specified by the ODMG Java standard):

```
import org.odmg.*;
import java.util.Date;

public class Invoice {
    Date      date;
    Customer  customer;
    SetOfObject items;
    /*...*/
}
```

The Relational Model

Database systems are designed for managing large amounts of data, and they provide many important features that object oriented programming languages do not, e.g.:

- fast queries
- sharing of objects among programs
- sophisticated error handling for database operations.
- permanent storage

Relational database systems (RDBMS) and record oriented systems based on B-Tree or Indexed Sequential Access Method (ISAM) are the standard systems used in many software developments. Each requires that all data be portrayed as a series of two-dimensional tables. The relational model defines the structures, operations, and design principles to be used with these tables. Languages like SQL provide ways of combining tables to express the relationships among data which are almost completely absent in most B-Tree or ISAM programming libraries.

During the 1980s relational databases became so popular that almost all producers of database software claimed that their products were relational. E.F. Codd responded by releasing a set of 12 rules that should be satisfied by any system claiming to be relational. Unfortunately, these rules excluded all systems then available, even systems like SQL which were based directly on Codd's relational calculus! Since the most powerful relational systems on the market are based on SQL, most of our discussion will focus on SQL database systems. However, everything we say about relational data structure also applies to other table oriented systems.

Relational Data Structure

Table Structure

In relational databases all data is stored in two-dimensional tables. These tables were originally called relations, which led to the name relational databases. An application will generally have a series of tables to hold its data. A database is nothing more or less than a collection of two-dimensional tables.

The rows of this table are composed of fields containing types known to the database. In most systems new data types can not be added.

Primary and foreign Keys

Each row in a table corresponds to one item or record. The position of a given row may change whenever rows are added or deleted. Items stored in the tables can be identified only by their values.

In order to uniquely identify an item it must be given a field or set of fields which is guaranteed to be unique within its table. This is called the primary key for the table, and is often a number. For instance, in our invoice table we might have a field called "invoice-number".

If one object contains the primary key of another object then this allows a relationship between the two items and is called a foreign key. For instance, in our simplified invoice example we might have the following tables:

Invoice

Invoice number	Customer number	Date
1276	1232	3 / 23 / 1992
1289	4567	4 / 1 / 1992

Invoice number is the primary key of the *Invoice* table. *Customer number* is a foreign key referring to a row in the *customers* table:

Customers

Customer number	Name
1232	Eddy Murphy
1456	George Bush

An invoice may contain many rows. Since a row in the invoice table can not hold a variable number of items we need to create a table called Invoice Rows which holds the individual items for an invoice. The primary key for this table contains two fields, Invoice Number and Row Number:

Invoice Rows

Invoice number	Row number	Quantity	Item number
1234	4567	123	1001
1234	1232	1	1032
1289	4567	15	2045

In order to access the price, name, and current stocking level of an item we need to look at the Items table:

Items

Item Number	Name	Price	Number in stock
1001	Elephant Soup	22.34	1066
1032	Nuclear Dental Floss	10.04	157

Data Manipulation

New rows are added to tables using the INSERT statement. Existing rows are changed or deleted using the UPDATE and DELETE statements. The SELECT statement performs queries. UPDATE, DELETE, and SELECT statements select rows based on their values, and operate on all rows in the table.

To add a new item to our table we could use the following code:

```
INSERT
INTO ITEMS (ITEM_NUMBER, NAME, PRICE, QUANTITY)
VALUES ('10036', 'Oil of Broccoli', '5.23', '100')
```

To add one dollar to the price of each item costing less than a dollar we could use this code:

```
UPDATE ITEMS
SET PRICE = 1 + ITEMS.PRICE
WHERE S.PRICE < 1.00
```

To delete all items which cost less than ten dollars we could use this code:

```
DELETE
FROM ITEMS
WHERE P.PRICE < 10
```

Rows from different tables may be combined using joins, which are part of the SELECT statement. The result of such a join is a row, but one that is not stored directly in one table. For instance, to see all item rows which contain Cream of Elephant Soup need to combine rows from two different tables. We could use this code:

```
SELECT INVOICE_ROWS.INVOICE_NUMBER,
       ITEMS.NAME, INVOICE_ROWS.QUANTITY
FROM INVOICE_ROWS, ITEMS
WHERE ITEMS.NAME = 'Cream of Elephant Soup'
```

Views

Virtual tables may be created by specifying the query that should be used to produce the table. These tables are called views, and may combine data from many tables. For instance, we may want to have a more useful form of an invoice row which contains the name and price of the item purchased, but without the inconsistencies that result if we duplicate data. We could do this with the following statement:

```
CREATE VIEW EXPANDED_INVOICE_ROW (INAME, PRICE, QUANTITY)
AS SELECT ITEMS.NAME, ITEMS.PRICE
INVOICE_ROWS.QUANTITY
FROM ITEMS, INVOICE_ROWS
```

Embedded SQL

Pure SQL is an interpreted language which is limited to database operations. Real applications are rarely written in SQL, but are written in other languages that generate SQL code or embed SQL code fragments passed to the database as ASCII text.

The SELECT statement returns a table of results, but most programming languages can not manage tables. Cursors are a method for accessing a table sequentially. A cursor is always positioned on a given row. It may be used to retrieve the current row or positioned to another row.

Combining the Models

Relational database systems and object oriented programming systems have very different programming models. They are not compatible. C++, for example, does not have any standard method for accessing data type information at run time, so your interface to the database is likely to be application specific. In Java there are interfaces (APIs) to access relational data storages (e.g. JDBC), however, the principles of these APIs are table-oriented, of course, which does not fit to the object oriented philosophy of the Java technology language.

Basic Strategies

There are a few general strategies that can be used. You can try to model the database in your object oriented program, you can try to model your application in the database, or you can simplify your database access to the point that the problems become less severe.

Model the Database in your Application Classes

One method is to build your object oriented application around the relational model. Each object needs to know how to store and retrieve its data from the database. This requires extra programming in every class,

and does not result in a clean program design since details of the database implementation must be reflected in every class. Since the data structures of object oriented languages are not closely related to the tables of relational databases this code is often non-trivial.

Model your Application in the Database

The second method is to reflect the object model in the relational database. This tends to be much harder than modeling the database in application classes since relational databases have limited data types, and it requires significant coding in the object oriented application because many aspects of objects can not be expressed directly in relational databases (e.g. inheritance, pointers and references, polymorphism, collections).

Row-oriented Database Interface

If your application only needs simple data types which do not have many relationships then you may be able to limit your interface to row-oriented access. This will greatly simplify your development. In many databases it may be useful to define views which present data to the program in a convenient way.

Some applications need to store complex data but exchange only simple data with a relational database. For such applications it may make sense to use an object oriented database for most data and use simple tables and views to share data with the relational database.

In the rest of this presentation we will show the difficulties involved in supporting an object oriented application with a relational database. We will focus on the problems encountered in data definition, data manipulation, and queries.

Database Schema Definition for Objects

The database schema refers to the structure of the database; in this case, the tables that should be created in the database to support our object oriented application. We will organize our discussion by the basic attributes of objects.

Encapsulation

The ability to combine code and data to form objects is basic to the organization of object oriented programs. Since relational databases do not support this you must write functions for each class to extract data and store it in the database. Similarly, you must also write functions to build objects and retrieve their data from the database.

Relational databases do not support implementation hiding except through password protection, but your whole program will generally run at one privilege level. Everything your program needs becomes public for your program. You can protect yourself somewhat by creating public and private views for the data members of a class, but this only helps if you remember which view to use in each part of your program. Instead of the protection your object oriented programming language guarantees you, you have to settle for conventions that will minimize the risk.

Inheritance

Since relational database systems can not store functions you must ensure that the function hierarchy is restored when you read objects from the database. This usually requires type information to be stored in the database to enable your application to build objects appropriately before loading their data.

The data hierarchy can be represented by creating a separate table for each point in the hierarchy. For instance, if your hierarchy includes an Item, a ClothingItem, and a LingerieItem then you need three tables, one for each class. Each LingerieItem must be stored three times to make sure is present in each of these tables.

Every time you process a LingerieItem you must ensure that all three tables are handled appropriately.

Polymorphism

Relational databases offer no support for polymorphism. You must be careful to store enough information about your data types to enable you to reconstruct objects properly before loading their data.

Object Identity

You can ensure that no two rows in a table have the same primary key by defining the field to be unique. This lets you use the primary key as a form of database identity. However, this does not prevent you from loading multiple copies of an object, which can cause inconsistent updates. In other words, the object identity in your database is lost as soon as the object is loaded into memory.

References among Objects

References among objects are expressed by foreign keys. In general, any reference in your objects will be expressed as a foreign key in your database table. The code you use to store your objects must follow object references and ensure that the proper foreign key is stored for each referenced object. If you intend to follow references in objects that have been retrieved then you must carefully follow all foreign keys, load the appropriate objects, and correctly initialize the object references.

Data Manipulation for Objects

In the previous section we have seen that relational databases can store only the data from objects and offers no support for many basic aspects of the object model. We have also seen that single objects generally must be stored as a series of tables to express the data hierarchy, and that the relationships among objects must be established using primary and foreign keys.

It comes as no surprise that this data model complicates data manipulation. We would like to illustrate this by describing the steps needed to store, retrieve, update, and delete objects in the database.

Storing Objects

Decide which Tables are used to store the Object

Most objects can not be stored in a single table, but need a series of tables to store the object hierarchy, arrays, variable length data, etc.

The documentation for each class should generally contain detailed information on how it is stored in the database, since this can be rather complex and is not always obvious.

Create Primary Keys for the Tables

Objects that are stored in relational databases should generally contain all primary keys as data members to make it easier to test whether keys have been assigned. Each object will need a primary key for each row in each table used to store it.

Since the database structure is not closely related to the class structure this tends to complicate your classes significantly, but you generally need some way to determine if and how an object has been stored in the database.

Update or create foreign Keys for referenced Objects

You need to convert each reference in your object to a foreign key since memory references have no meaning in the database.

Extract Data and store it

Do this within a transaction to ensure that the whole object is stored if the transaction succeeds. Since only the object itself knows how to find its private members the object should extract its own data, and it may make sense to have the object store its data.

Store referenced Objects, if necessary

Depending on the semantic relationships among your objects you may want to store all referenced objects to ensure a consistent state. If so, this may be in the same transaction or a different transaction than the one used to store the original object.

If your objects contain a dirty flag then you can avoid unnecessary database operations when storing referenced objects.

Associate Memory References with Keys

You probably want to avoid multiple copies of the same object in memory, which violates the semantics of the object model and can lead to a variety of update consistency problems. This means that you need a way to determine if an object in the database has already been loaded. The best way to do this is to keep a table in memory which associates the primary keys with the memory references of existing objects. You have to develop a complete object management system for your memory.

Retrieving Objects

Read Type Information to see how Object is built

You need to know the entire class hierarchy of an object to build it correctly. If a ClothingItem is stored then you can read it as an Item, but it will not have a size and will not know how to compute sales tax correctly to determine its total price.

Determine if the Object is already in memory

If you have a table in memory which associates primary keys with the memory references of existing objects then you can use this to determine if the object is in memory.

If the object is in memory then you generally return a reference to the existing object rather than creating a second copy. Since this is the program's working copy you probably don't want to overwrite its values with those stored in the database.

Build a new Object, if necessary

If you have stored the complete type information for an object then you can call the appropriate constructor to create a vanilla object that can be stuffed with data. This object will have the right methods, so polymorphism will be preserved.

Fill the new Object with Data from the Database

If your object contains private members, it will have to do this itself.

Load referenced Objects, if necessary

If your object contains references to other objects, then you will generally want to load the referenced objects and initialize the references to the corresponding objects. This prevents your program from throwing an exception when it tries to follow a null reference or an uninitialized reference.

Changing Objects

Changing objects is similar to storing objects, but you will need the primary key of each table to allow you to specify the UPDATE operations for your database.

Deleting Objects

When deleting objects make sure that the object is correctly deleted in each of its tables. Do this in a transaction to make sure that the entire operation succeeds or fails as a unit.

Queries

Specifying the criteria for a query is somewhat tricky because you have to use the semantics of the data in the object and the structure of your tables to create SQL code for your database. To do this you will need

to know which table is used to store each data member and how the primary and foreign keys are used to build the relationships among the object's data.

This becomes even trickier if you want to do a query that involves values in a referenced object. Now two objects need to cooperate to build the query since neither object has knowledge of the database structure of the other object.

Since the result set of a query is a full table you need to find some way to bring results into your query. You will need to use cursors to step through the results sequentially. In some cases you may want to return key values for the objects which have been found, in others you may want to load the objects and return references to them. Note that you have to avoid creating multiple copies of objects in memory when you perform queries!

Limitations of These Solutions

We have tried to show how to support objects with a relational database system. None of the operations we have described are particularly complex; these are meat-and-potatoes operations that will be used in most programs. Although the operations are not complex, the interface needed to support them is reasonably complex, and it has several significant shortcomings:

Two semantic Spaces, Two separate Designs

Relational databases and object oriented programming languages have fundamentally different programming models, and you need to create two different data models for your application. After you create your class hierarchy you need to decide how each class should be stored in the database.

Complex Interface between the two Designs

Because the relational model does not support the most basic aspects of the object model the interface to the database has become complex. The reason is simple: relational databases were never designed to store objects, and objects were never designed to be stored in two dimensional tables.

This Interface is reflected in each Object

A complex interface is bad enough, but when this interface has to be reflected in every single object then it becomes much more difficult to maintain. The result is that your whole program tends to revolve around the fact that you intend to store and retrieve these objects in a database.

All real Object Operations must be Implemented in your Program, not the Database

Your program must build objects when reading their data from the database. It must store all type information and create appropriate constructors for each class that can be retrieved from the database, and it should generally maintain tables to tell it which objects have been loaded. Each object must know how to store itself, and building queries becomes complex.

Object Oriented Databases

Because many people have become frustrated when storing objects in relational databases, a new kind of database has been developed which supports object oriented programs directly.

Support for Objects

There are some databases which only pretend to be object oriented, but real object oriented databases provide full support for objects. Full support of objects, e.g., includes the programming language and type system (e.g. Java technology or C++), the transaction model, queries, and finally the storage model.

Encapsulation

An object which is read from the database has the same code and data that it had when it was stored. Objects may have private and protected parts, and they are managed appropriately.

Inheritance

Classes which are derived from other classes can be stored with one operation. The database system must know the class hierarchy and manage object storage accordingly.

Polymorphism

When an object is read from the database it is given all the code and data members that it had when it was originally read. This is true even if you read it without knowing its complete type information. For instance, if you store a rocking chair, you might read it back when you look at all the chairs in your database. If you do so, the chair you just read can rock.

Object Identity

Object oriented database systems integrate the object identity in the database with the identity of objects in memory. If you store an object then it knows if it corresponds to an object in the database, and when you retrieve an object it knows if the object has already been loaded into program memory. There is no need for the programmer to maintain the relationship between database objects and objects in memory.

References among Objects

True object oriented database systems can automatically resolve references in your program's objects and represent them in the database.

Basic Database Operations

We would like to show you how basic database operations can be performed in object oriented database systems. In order to be concrete we will provide examples using FastObjects™ by Poet, a commercial object oriented database system which runs on Windows, Linux, and UNIX systems. This is not an overview of FastObjects™, which has many more features than we can cover in a few pages, but simply a way of illustrating that basic database operations are much simpler with an object oriented database system.

For the world of Java technology, the ODMG Java standard specifies how Java objects can be stored in databases. The key principles behind this truly object oriented standard are “transparent persistence” and “persistence by reachability”.

“Transparent persistence” means that application developers do not have to distinguish between database operations and other operations on their objects. The Java technology language and the database system share the same type system – the same type system is used by both, persistent and transient objects. In addition, both the syntax and the semantics of the Java language are retained. No additional language components (keywords) for defining persistence-capable classes are involved. The class definitions for transient classes and persistence-capable classes are identical, thus freeing developers from having to code explicit database operations for storing and retrieving objects.

Storing Objects

“Persistence by reachability” is another key concept in the ODMG Java technology standard. When a transaction is committed, this feature ensures that not only those objects that were made explicitly persistent (so-called persistent root objects) are written to the database, but all referenced objects as well. In other words, every object that can be reached directly or indirectly by a reference from a persistent root object also becomes persistent itself. The item under consideration is thus no longer the individual object but, rather, logically related object networks: entire object networks become persistent, not just individual objects. Root objects serve merely as starting points. In practice, this results in very simple, clear code.

If an object has been bound (via `bind()`) to a database, the transaction transparently stores the object along with its related objects to the database (`commit()`).

```
//...
Database database = new Database();
database.open(url, Database.OPEN_READ_WRITE);
//...
Transaction txn = new Transaction();
txn.begin();
try {
    MyObject o = new MyObject();
    //...
    database.bind(o, "root");
}
//...
txn.commit();
```

Changing Objects

Changing objects is just as simple as initially storing objects.

Objects are changed by changing them in memory and letting the transaction transparently store them back to the database inside its `commit()` method. Because the transaction knows the identity of all stored objects it can tell whether the object exists in the database or not; if so, it updates the existing object instead of creating a new one.

The transaction is one of the pivotal concepts in the ODMG Java technology standard: operations on persistent objects must always be performed in the context of transactions. A transaction is an all-or-nothing operation. It takes the database from one logically valid status to another. The database system itself guarantees that interim statuses remain invisible to other database users. At the end of a transaction, the application calls either “commit”, which permanently affects all changes in the database, or “abort”, which discards all changes made since the beginning of the transaction.

Deleting Objects

Java technology systems delete objects in memory through the built-in garbage collection mechanism. Isolated objects (objects that are not referenced by any other object in the system) are assumed to be useless and can just be deleted. A garbage collector agent detects such objects and deletes them transparently.

With database systems this issue is a bit more delicate. You do not really want a mysterious garbage collector going ballistic in your database system at unforeseeable times and deleting objects. Therefore database objects are deleted either explicitly through API calls or through an according database administration facility.

The ODMG Java standard does not specify how database objects are to be deleted. The FastObjects™ implementation of this standard, however, provides a special “service” that can be used to delete database objects explicitly. In addition, an external garbage collector is provided.

Queries

How do you find a particular object in the database? One way of locating a specific object in the database is by executing a query. The ODMG standard specifies the Object Query Language (OQL) as the common query language for all supported language bindings. OQL can be seen as the object oriented equivalent of SQL.

Queries support all the semantics of your objects and objects to which they refer. Although the details of OQL are not discussed here, a brief example is provided: Search for all people in the database whose names start with the letter 'M'.

Queries Continued...

```
SELECT Person
FROM PersonExtent AS Person
WHERE Person.name="M*";
```

In addition to interactive queries, OQL queries can be submitted via objects of the class `OQLQuery` or to collections.

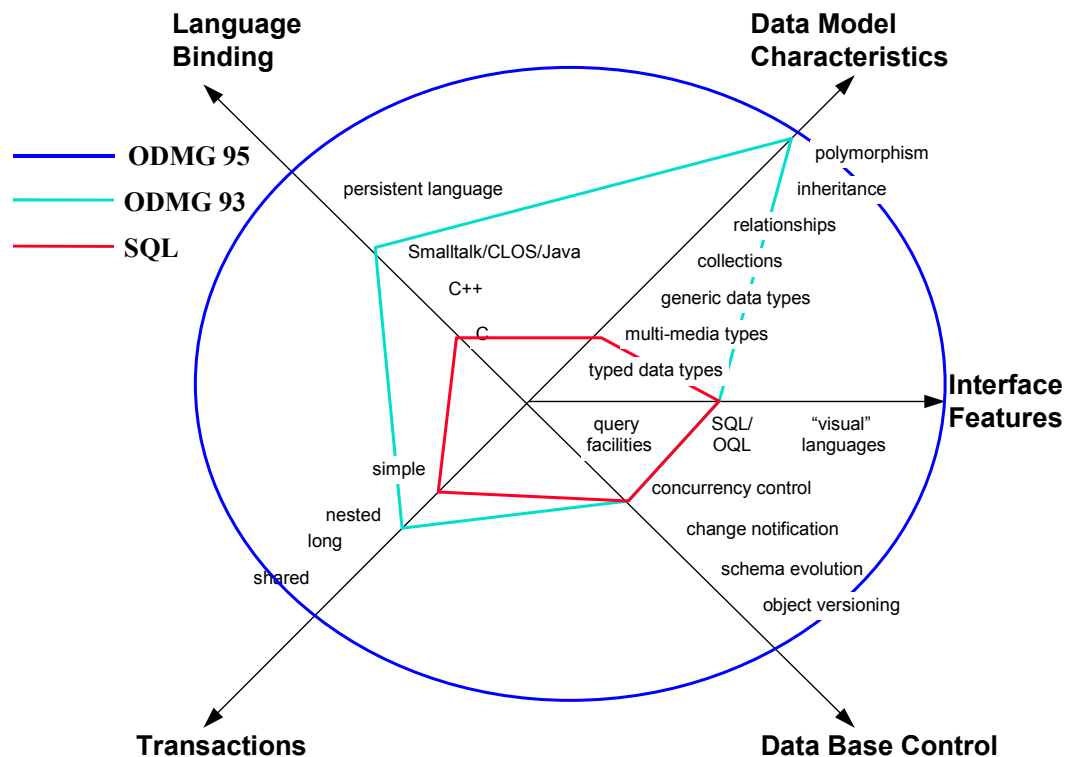
```
//...
OQLQuery query = new OQLQuery();
query.create(
    "SELECT Person " +
    "FROM PersonExtent AS Person " +
    "WHERE Person.name=\"M*\";");
Object result = query.execute();
```

Summary

The relational model is simple and elegant, but it is fundamentally different from the object model. Relational databases are not designed for storing objects, and programming an interface for storing objects in a relational database is not trivial.

We have shown that relational databases are poorly adapted to express the basic attributes of objects like encapsulation, inheritance, polymorphism, object identity, references among objects, and user defined data types. The ramifications of this have been illustrated by showing what your program must do to correctly handle simple operations like store, read, delete, change, and search for objects. The strategies we illustrate may be useful for those who need to interface object oriented applications to relational databases.

Finally, we have shown that object oriented databases greatly simplify object database management because they support the object model directly. We have illustrated this by using some simple ODMG code programmed for FastObjects™, an object oriented database for Java technology and C++.



Comparison of SQL and ODMG standards

The ODMG standard specifies how objects are stored in database systems. The Java technology part of the standard, for example, specifies how database systems communicate with the Java™ programming language, providing an intuitive, object oriented Java technology interface that allows Java™ objects to be stored easily and efficiently.

The first release of the ODMG standard was published in 1993. The latest release 3.0 was published in the year 2000. Today the ODMG standard is the state-of-the-art of object database programming, especially for Java™ applications. FastObjects™ by Poet is one of the most successful implementations of the ODMG Java™ standard.